LLM-Based Test Generation: Information Sources, Generation Strategies, and Quality Evidence

ANONYMOUS AUTHOR(S)

Large language models are increasingly used to generate test scenarios, executable test suites, assertions, interaction sequences, and fuzzing infrastructure. These artifacts serve different purposes and rely on different sources of information about correct behavior. A test that increases implementation coverage, a test that agrees with a reference program, and a test that detects a requirements violation therefore provide distinct kinds of evidence. We present a structured narrative survey that connects the generation process to the evidence used to justify test quality. Drawing on a curated literature collection through September 2026, we organize the field along four dimensions: testing objectives and artifacts, information available during training and generation, generation and learning mechanisms, and evaluation evidence. We synthesize work on unit and requirements-based testing, test oracles, API and GUI testing, fuzzing, and learned test generators. The resulting framework explains how execution feedback can improve executability while also affecting the independence of an oracle, why training-time reference access matters for comparisons, and how downstream code-selection gains differ from test correctness. We use these distinctions to organize benchmarks and derive a research agenda for independent oracle assessment, budget-aware generation, repository-scale evaluation, and test maintenance.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Natural language generation.*

Additional Key Words and Phrases: large language models, test generation, test oracles, software testing, survey

ACM Reference Format:
Anonymous Author(s). 2027. LLM-Based Test Generation: Information Sources, Generation Strategies, and Quality Evidence. In *Proceedings of FSE 2027*. ACM, New York, NY, USA, 21 pages.

1 Introduction

Test generation requires a connection between what software should do and observations that can expose when it does something else. Constructing that connection involves several kinds of work: interpreting requirements, selecting inputs, arranging an execution environment, choosing observations, and expressing an expected result. Large language models (LLMs) can contribute to each of these activities because they operate across natural language, program code, and semi-structured technical documentation. Research has consequently expanded from generating unit-test code to producing acceptance scenarios, postconditions, API call sequences, and fuzzing drivers [50, 56, 58].

This breadth creates a comparison problem. Two approaches can both generate executable tests while having different access to the system under test (SUT), different sources of expected behavior, and different intended uses. A regression generator may inspect an implementation and retain tests that pass on it. A requirements-driven generator may need to produce an oracle before an implementation exists. An issue-reproduction agent may receive a report of an already observed failure. The same passing test carries a different meaning in each setting. Comparing their coverage or execution success without describing these conditions obscures what has actually been learned about test quality.

Recent systems make this distinction especially important. TestGen-LLM improves existing tests through operational filters, including execution and coverage checks [2]. CoverUp uses coverage

FSE 2027, Shenzhen, China
2027.

feedback to guide Python test generation [38]. In a different setting, PyTester learns to produce tests from textual descriptions, using executable reference information during training [46]. UTRL couples test and code generation through adversarial reinforcement learning [26]. These systems differ in where behavior is observed, when it is available, and how the observations influence the final test. Their common use of an LLM does not determine their testing objective.

Existing surveys provide substantial foundations. Broad reviews map LLM applications across testing and repair [4, 50]; unit-testing surveys organize techniques, model adaptation, and quality assurance [10, 58]; requirements-based reviews cover transformations from natural-language and formal specifications [18, 56]. A recent systematic review directly addresses LLM-based test-case generation [47]. Oracle generation, hybrid techniques, and evaluation limitations already appear in these accounts. Our organizing question is how these aspects interact: *what information produces a test, and what evidence justifies using that test to judge software?*

We develop this question through a structured narrative synthesis of a curated collection of 60 source records. The collection includes seven surveys and roadmaps, together with primary methods, benchmarks, empirical studies, and selected neural predecessors. Its breadth supports a cross-setting analysis; its preliminary screening status is described in Section 3. We make three contributions. First, we define a framework that separates testing objectives, generated artifacts, information access, and quality evidence. Second, we use the framework to connect prompting, program analysis, feedback, agents, and model training across testing settings. Third, we organize evaluation designs according to the claims they can support and derive concrete research questions from the remaining mismatches.

The central argument is that test generation should be understood as both an artifact-construction problem and an evidence-construction problem. Better syntax and broader execution improve the first. Confidence in expected behavior requires the second. This distinction helps explain when tools from different research traditions can be combined and which claims require additional validation.

2 Foundations and Analytical Framework

2.1 What Is Generated?

An executable test combines a setup, an input or action sequence, and an oracle that judges the resulting behavior. Its setup can include fixtures, mocks, dependencies, credentials, database state, and cleanup. Its input may be a function argument, an HTTP request, a GUI action, or an entire generated program. Its oracle may check an exact output, an exception, a relation across executions, or a safety property. A test suite additionally determines which tests coexist, how state is isolated, and when execution ends.

Many generation methods produce an intermediate artifact. A natural-language scenario still requires operationalization. A grammar defines a family of admissible inputs but does not necessarily determine their expected outputs. A fuzzing driver connects generated data to a target API while relying on a separate failure detector. We therefore distinguish *test design*, *test input*, *test oracle*, *executable test*, and *test-generation infrastructure*. This distinction preserves the contributions of input-only and infrastructure methods without attributing complete test synthesis to them.

2.2 Testing Objectives Determine Oracle Roles

Functional conformance asks whether behavior satisfies an external requirement. Regression testing asks whether a change violates behavior that should be preserved. Bug reproduction asks whether a reported failure can be exercised and distinguished from its repair. Fuzzing often searches for crashes, sanitizer violations, or disagreement among implementations. Tests used in code generation

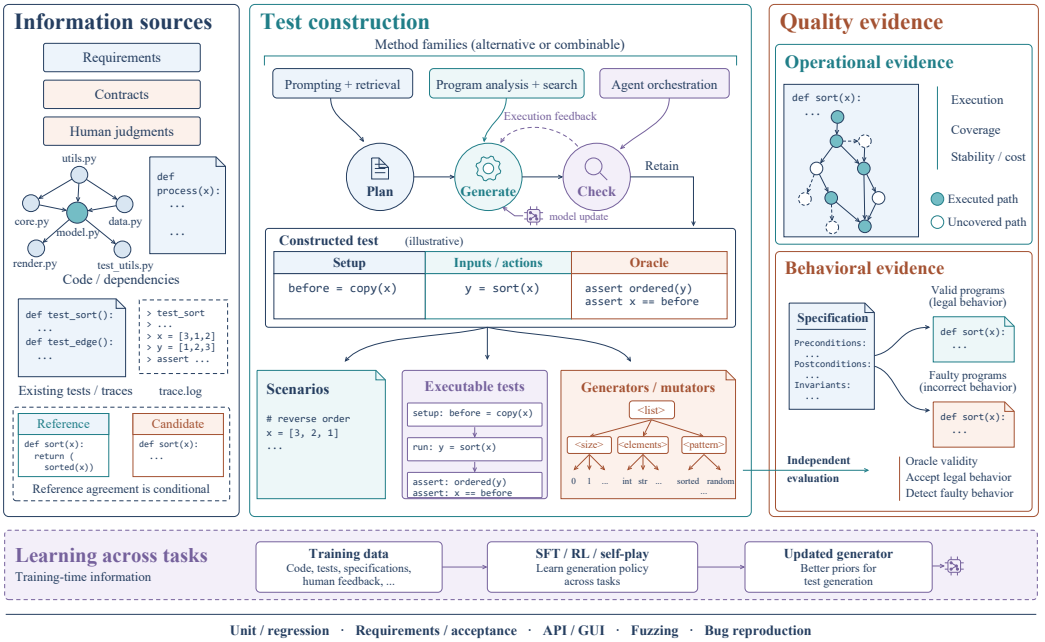


Fig. 1. A synthesis of LLM-based test generation. Information sources support planning, generation, and checking; alternative or combined mechanisms construct scenarios, executable tests, and generators. The sorting sketch illustrates setup, action, and a partial oracle. Dashed feedback revises the current artifact; matching violet model icons connect the cross-task learning output to generation. Operational and behavioral evidence address different quality properties. Independent evaluation supplies held-out evidence without feeding its results back into generation.

may instead rank candidate solutions or provide debugging feedback. These objectives can share execution machinery while requiring different oracle evidence.

Consider a requirement that a function return a sorted permutation of its input without modifying the input. A test may assert the sorted result yet omit the mutation constraint. It can execute successfully and reach every line while leaving part of the requirement unchecked. A test copied from an implementation that sorts in place can even preserve the very violation that a conformance test should reveal. Conversely, a test that expects one particular ordering among equal-key objects may reject a legal implementation if stability was never required. This illustrative example motivates evaluating both detection of invalid behavior and acceptance of permitted behavior.

2.3 Information Access and Correctness Evidence

We distinguish three stages of information access: training, generation, and final evaluation. At each stage, relevant sources include requirements, focal code, repository context, existing tests, reference implementations, candidate implementations, execution traces, and human decisions. Access is a property of an experimental variant rather than an entire paper. For instance, inference can use a description and signature even when training rewards require a reference program, as in PyTester [46]. Describing that configuration simply as “code-free” would erase a material dependency.

An information source can play more than one role. Source code helps resolve imports and construct objects; the same code can also supply expected outputs. A static dependency graph can identify callable APIs without validating their semantics. Execution of a trusted reference provides a concrete output under the reference assumption. Execution of the candidate under test provides an observation about that candidate. These roles should be recorded separately because context relevance and oracle independence answer different questions.

Figure 1 connects the four dimensions of our analytical framework: objectives and artifacts, information access, generation mechanisms, and quality evidence. The dimensions are intentionally orthogonal. "Agent-based" describes how decisions are coordinated; "requirements-based" describes an information source; "unit testing" describes a testing level. Treating these labels as competing branches of one taxonomy would place overlapping studies in arbitrary categories. We instead use testing settings to organize the landscape and mechanisms to explain recurring design choices.

3 Review Scope and Synthesis Method

3.1 Research Questions

Six questions guide the synthesis and connect its sections:

RQ1: Objectives and artifacts. Which testing objectives, systems, and levels are addressed, and what artifacts are generated? Section 4 maps the settings and outputs.

RQ2: Information and oracles. What information is available during training and generation, and where does evidence of correct behavior originate? Sections 2 and 6 distinguish access stages and sources.

RQ3: Generation strategies. How are LLMs combined with retrieval, analysis, execution, search, and agents? Section 5 compares these mechanisms.

RQ4: Learning objectives. What do training data, rewards, and adversarial curricula optimize? Section 6 examines learned generators.

RQ5: Evaluation evidence. Which quality claims do benchmarks support, and under what conditions are results comparable? Section 7 analyzes protocols and measurements.

RQ6: Practical use and open problems. What is supported by development-workflow evidence, and which capabilities still need validation? Section 8 connects adoption, maintenance, and research directions.

3.2 Collection and Selection

This survey is a structured narrative synthesis of a preliminary literature collection assembled through 28 September 2026. Discovery used public web search, primary publication pages, arXiv records, author-hosted resources, and references identified through existing surveys. Query families combined large language models with unit-test generation, requirements-based testing, oracle generation, API and GUI testing, fuzzing, reinforcement learning, and evaluation benchmarks. Named-method searches expanded these families to identify related versions and recent developments. The source register contains 60 records, including seven surveys and roadmaps; these counts describe the curated register rather than an exhaustive population of studies.

We retained sources that describe an LLM or a relevant neural predecessor producing a testing artifact, or that directly evaluate such generation. Work on generated tests for debugging and code selection was included when it clarifies the role of test quality in a downstream task. General code generation and research concerned solely with testing an LLM itself fall outside the central scope. Selected earlier neural approaches provide historical context for focal-code conditioning and learned oracles [14, 48]. Publicly accessible primary sources were preferred over secondary

Table 1. Existing review families and the emphasis of this synthesis. These are differences in scope and organization, not claims that individual topics were previously absent.

Review family	Established coverage	Emphasis here
Broad testing surveys and roadmap [4, 50]	Testing tasks, LLM techniques, challenges, repair-related applications	Connect generated artifacts to their oracle evidence
Unit-testing surveys [10, 58]	Generation methods, model adaptation, quality assurance, lifecycle concerns	Compare code-conditioned and specification-conditioned settings
Requirements-based reviews [18, 56]	Requirements representations, transformations, industrial use	Separate scenario design, input construction, and executable oracles
LLM test-case SLR [47]	Techniques, data, metrics, preprocessing and postprocessing	Track information access across training, inference, and assessment

summaries. The initial retrieval excluded MDPI publications and sources with unclear provenance, which constrains source coverage.

Related preprint and publication versions were treated as a study family. A changed title alone was not counted as a new method. The bibliography cites a verified publication record where available and otherwise identifies the accessible preprint. Publication status and evidence strength remain separate: a peer-reviewed abstract can establish a method's broad objective, while its detailed information access may still require inspection of the full text.

3.3 Extraction and Interpretation

The source register records the artifact, information sources, principal mechanism, analytical use, and depth of inspection for each entry. All records were checked against a primary-source page or document. Most were examined at the abstract and metadata level; selected full-text passages were inspected for survey taxonomies, oracle feedback, training rewards, and benchmark definitions. We use those sources for qualitative mechanism descriptions and do not pool effect sizes or construct a cross-paper performance ranking. No independent dual screening, inter-rater reliability measurement, or reproduction of reported experiments has been performed for this collection.

The synthesis proceeds by separating three kinds of statements. Method descriptions explain what the cited work constructs or optimizes. Evaluation interpretations identify which property a reported protocol measures. Research recommendations follow from comparing those properties across settings. For example, the observation that a method checks a reference implementation is a description; the conclusion that this check establishes reference consistency is an interpretation; testing additional legal implementations is a recommended evaluation extension. Keeping these roles distinct makes the reasoning auditable without treating every recommendation as an empirical finding.

AI assistance was used to expand search terms, organize retrieved literature, propose the cross-study categories, draft the synthesis, and design its conceptual figures. The accompanying register preserves source links and inspection depth. The present coding has not received independent human validation. This status is relevant because category assignments can affect the survey's conclusions even when individual bibliographic records are correct.

Testing setting	Scenarios / obligations	Inputs / actions	Tests / oracles	Generators / mutators
Unit / regression			TestPilot [P,F] CoverUp [P,F]	
Requirements	RAGTAG [P,R]		Test Flow [P]	
Oracle synthesis			nl2postcond [P] Nexus [P,F]	
API / GUI		DroidAgent [P,F]	SAINT [P,A]	
Fuzzing		TitanFuzz [P]		PromptFuzz [P,F] ELFuzz [P,S]
Code-task testing			PyTester [L] UTRL [L]	SAGE [L]

Mechanisms: P prompting; A analysis; R retrieval; F execution feedback;
L parameter learning; S search. Blank cells are unlisted, not absent.

Fig. 2. A representative literature map, based on the methods discussed in this section [1, 3, 8, 12, 15, 17, 21, 26, 31, 36, 38, 41, 46, 57]. A setting can produce several kinds of artifacts, and mechanisms can be combined. Blank cells indicate no example selected for display.

3.4 Relationship to Existing Reviews

Table 1 places this account alongside existing reviews. Its intended value is the connection between dimensions: how a feedback source affects oracle independence, how the generated artifact changes the evaluation denominator, and how training supervision constrains claims of autonomy. These connections also provide a common vocabulary for integrating 2026 work on adversarial training, agents, and test maintenance into the earlier literature.

4 Testing Settings and Generated Artifacts

Figure 2 maps representative work to testing settings and generated artifacts. The mechanism labels identify selected operations discussed below; they are illustrative assignments rather than exhaustive coding or publication counts.

4.1 Unit Tests and Repository Context

Unit-test generation is the most visible setting in the reviewed collection. The task appears local because the target is a function or method, but executable tests depend on surrounding types, imports, initialization, and test conventions. AthenaTest established an earlier transformer-based approach that uses focal context and supervised code–test data [48]. TestPilot combines function information and documentation with execution-based repair [41], while ChatUniTest organizes generation, validation, and repair into a framework [9]. These approaches illustrate the transition from generating a plausible test body to constructing an executable artifact in its environment.

Repository context becomes particularly important when the target cannot be instantiated in isolation. CITYWALK uses project-dependency awareness and language-specific knowledge for C++ test generation [60]. Its setting highlights why results on standalone functions do not

directly characterize class construction, linking, fixtures, or external dependencies. In regression-oriented systems such as TestGen-LLM, existing tests additionally supply conventions and a baseline to improve [2]. These conventions can make generated tests easier to integrate, although their behavioral assumptions still require interpretation.

4.2 Requirements, Scenarios, and Acceptance Tests

Requirements-driven generation begins with intended behavior expressed independently of an executable test. RAGTAG retrieves domain information to generate test scenarios from natural-language requirements [3]. Multi-step generation of system-level test specifications uses intermediate artifacts to structure this transformation [34]. Both settings expose an important distinction: producing a useful test design and producing a runnable script are separate accomplishments, with different evaluation methods and human responsibilities.

An industrial acceptance-testing study makes this separation explicit. AutoUAT transforms user stories into Gherkin scenarios, while Test Flow uses those scenarios and page HTML to construct Cypress scripts [17]. Information access changes between stages. The scenario can be judged against a user story; the script also needs knowledge of concrete page structure. APITestGenie similarly combines requirements and API specifications to generate web API tests [37]. These pipelines suggest that traceability should connect the requirement, the scenario, the executable actions, and the assertion, so that a failure can be attributed to the appropriate transformation.

4.3 Oracles, Postconditions, and Metamorphic Relations

Oracle generation isolates the part of a test that decides whether behavior is acceptable. TOGA generates exception and assertion oracles using a neural approach [14]. Its subsequent large-scale evaluation and replication work demonstrates the importance of examining oracle generation under changed evaluation conditions [20]. An oracle should therefore be assessed by the behavior it accepts and rejects, in addition to whether it resembles a developer-written assertion.

Postconditions and metamorphic relations broaden the oracle representation. The nl2postcond work translates natural-language intent into executable method postconditions [15]. Work on executable metamorphic relations derives relations from requirements and API specifications [42]. A relation can test consistency across executions when predicting one exact output is difficult. Its applicability conditions are part of its meaning: a plausible transformation can be invalid for inputs that violate an unstated precondition. Thus, generating the relation and establishing its domain of validity remain distinct tasks.

Nexus uses execution-grounded multi-agent processes for oracle synthesis [21], and CANDOR incorporates multi-agent consensus into end-to-end JUnit generation [55]. These approaches combine test construction and oracle review. Their interaction patterns can expose contradictions, but the source of behavioral evidence still determines what agreement establishes. Agreement among agents or with candidate executions must be interpreted relative to the requirements being tested.

4.4 API and GUI Interaction Sequences

For APIs and GUIs, test generation must maintain state across actions. An endpoint can require a resource identifier returned by an earlier request; a GUI action can depend on a screen reached through navigation. AutoRestTest combines LLM support with multi-agent reinforcement learning for REST API testing [44]. SAINT uses program analysis and LLM-based agents for service-level integration tests [36]. The relevant artifact is therefore a coordinated interaction sequence, including dependencies between operations, rather than a collection of independent input values.

DroidAgent and GPTDroid apply LLM reasoning to mobile GUI exploration and interaction [30, 57]. Here, observations and actions are interleaved: the next action depends on the current interface. Reaching a screen demonstrates exploration; checking that the screen implements a requirement additionally needs an oracle. This distinction also affects replay. A sequence generated through adaptive interaction may need stable selectors, state initialization, and checkpoints before it becomes a repeatable regression test.

4.5 Fuzzing Inputs and Infrastructure

Fuzzing gives the LLM a different role. TitanFuzz and FuzzGPT generate programs for testing deep-learning libraries, with the latter emphasizing edge cases [12, 13]. Fuzz4All uses LLMs in a broader fuzzing framework [53]. These methods exploit the model's ability to produce structured inputs while relying on execution and external failure detectors to identify anomalies.

The generated object can also be an input generator. ChatAFL extracts protocol-related information to support grammar construction and message-sequence exploration [33]. PromptFuzz generates fuzzing drivers [31]; ELFuzz searches over synthesized fuzzers [8]; Bulbasaur generates customized mutators using branch-oriented information [52]. This progression changes the cost model because one generated artifact can produce many executions. A survey that records only the number of LLM-generated test cases would miss both the reuse and the contribution of the conventional fuzzing engine.

4.6 Bug Reproduction and Tests for Code Generation

LIBRO generates bug-reproducing tests from natural-language reports [24]. SWT-Bench evaluates test generation in the context of real-world fixes [35]. These settings provide a particular behavioral distinction: the test should expose the issue in a buggy version and accept a fixed version. Reproducing a known failure and discovering an unknown defect are related capabilities but use different information.

Generated tests also influence which code is selected or how it is repaired. CodeT uses generated tests to help choose among candidate programs [7], UTGen learns tests for automated debugging [39], and CodeRM scales test generation for code reward modeling [32]. TiCoder introduces user feedback on tests to clarify intent during code generation [16]. In all four cases, the downstream outcome depends on both the generator and the consumer of its tests. A useful survey must therefore retain test-level measurements alongside code-selection or repair outcomes.

5 Generation Mechanisms

5.1 Context Construction and Task Decomposition

Prompting determines which parts of a testing problem the model sees at once. Direct prompts usually combine a target, an instruction, and a desired output format. Examples can provide the test framework, naming conventions, and assertion style. Context construction becomes a substantive method contribution when it selects information needed to resolve dependencies or infer the intended behavior. TestPilot and ChatUniTest illustrate focal-code and environment-aware construction [9, 41]; RAGTAG illustrates retrieval of domain material for requirements interpretation [3].

Retrieval has two separable functions. It can supply technical facts needed for execution, such as an API signature, or behavioral facts needed for an oracle, such as a business rule. These should be evaluated independently. An increase in compilation success can establish that retrieved API facts were useful without establishing that the retrieved business rule was current or applicable.

Useful ablations would hold the model and budget constant while removing each kind of retrieved context in turn.

Decomposition can expose intermediate decisions. SymPrompt structures coverage-guided prompting around execution paths [40]. Requirements pipelines separate interpretation, scenario construction, and script generation [17, 34]. Decomposition is particularly helpful for analysis because it makes failures attributable: an omitted scenario differs from an invalid locator or a wrong expected value. At the same time, intermediate artifacts can transmit an early misunderstanding to every later stage. Their value should be evaluated by final test behavior together with stage-specific error rates.

5.2 Input, State, and Environment Construction

Input construction must satisfy both the external input domain and the execution interface. Grammar-based methods such as SAGE and LogiCase make admissible input structures explicit [1, 45]; repository-oriented approaches must also recover fixtures and dependencies [60]. For API and GUI testing, validity extends to action order and state: a well-formed request can still be unusable before a required resource is created [36, 44, 57]. These mechanisms solve different parts of reaching a meaningful observation.

The sorting example in Section 2 illustrates this separation. A valid input can exercise duplicate keys, while setup must preserve a copy of the original collection so that input mutation remains observable. Generating sorted expected values alone would miss this setup obligation. Evaluations should therefore distinguish rejected inputs, failed setup, and assertion failures instead of combining them into one unsuccessful-test count.

5.3 Oracle Construction and Representation

An oracle can predict an exact result, check a property, relate several executions, or obtain a human judgment. Postconditions and executable metamorphic relations express behavioral constraints [15, 42]; TiCoder obtains user feedback to clarify intended behavior [16]; Nexus and CANDOR use agent interaction in oracle construction [21, 55]. The representation determines which behaviors the test can express, while the evidence source determines why those behaviors should be accepted.

For sorting, checking ordering, permutation preservation, and unchanged input directly represents the stated obligations. An exact sequence can add an unintended stability requirement, whereas a relation between executions needs its own applicability conditions. This connects oracle representation to legal-behavior non-rejection. Agent consensus can help inspect the proposed assertion, but its semantic validity still depends on those obligations and supporting evidence.

5.4 Program Analysis and Hybrid Search

Program analysis provides explicit structure that the model would otherwise need to infer from text. Dependency information can identify construction requirements; call relationships can constrain operation sequences; coverage can identify unexercised branches. CITYWALK and SAINT illustrate different uses of program structure for executable testing [36, 60]. The source of structure matters: a graph recovered from implementation describes reachable operations, while a behavioral contract specifies what those operations should mean.

CODAMOSA combines search-based test generation with LLM-generated examples when coverage progress stalls [27]. This division of labor is different from asking an LLM to generate an entire suite repeatedly. Search performs repeated local exploration, and the model supplies candidates that may open new regions. Fuzzing-driver and mutator generation extend the same principle to reusable generators [8, 31, 52]. These systems should be analyzed by the interaction between

components: when the LLM is invoked, which artifacts it changes, and how long those artifacts remain useful.

Hybrid generation creates evaluation choices that can change the interpretation of a result. A stronger search engine, more execution time, or additional instrumentation can improve outcomes even when model-generated artifacts are unchanged. Comparisons should therefore account for both model cost and conventional execution cost. Component ablations should preserve the rest of the pipeline, including its stopping rule, so that a benefit can be attributed to the LLM contribution or the hybrid interaction.

5.5 Execution Feedback and Iterative Repair

Execution feedback ranges from syntax and import errors to coverage reports and assertion outcomes. These signals constrain different parts of a test. A syntax error localizes a malformed artifact. A missing dependency can reveal incomplete environment construction. An uncovered branch supplies an exploration target. An assertion failure is more ambiguous because the cause can lie in the test, the implementation, the environment, or the interpreted requirement.

CoverUp uses coverage and execution information in iterative Python test generation [38]. TestGen-LLM filters proposed improvements through operational criteria that support regression-test integration [2]. MuTAP uses mutation testing to guide generation [11]. These systems demonstrate that feedback can be used for different decisions: repair a candidate, reject it, request another test, or redirect search. Recording only that a system is "execution-guided" loses this distinction.

For conformance testing, the decision following an assertion failure deserves special attention. Replacing the expected value with the candidate's observed output produces agreement, but it can also remove the discrepancy that the test was intended to expose. A more informative repair process distinguishes executable setup from semantic assertions. It can repair an import immediately while preserving an assertion until a requirement, trusted reference, or human decision explains the mismatch. This is a design implication of oracle provenance, rather than an assumption that all current feedback systems behave identically.

RESTestBench makes the quality of SUT feedback an explicit evaluation issue by studying tests generated from requirements under different SUT conditions [25]. This setting helps connect feedback research to requirements-based testing: the value of a repair signal depends on whether the observed behavior is itself valid. Paired evaluations starting from the same initial test would reveal both recovery of invalid tests and degradation of valid tests.

5.6 Agents, Collaboration, and Adversarial Generation

Agent architectures assign responsibilities such as planning, code generation, execution, and review to separate components. TestAgent follows a workflow inspired by human testing and incorporates static knowledge and tool use [59]. CANDOR and Nexus emphasize collaboration around test oracles [21, 55]. The analytical question is what separation changes: available information, decision authority, search diversity, or execution budget.

Role separation alone does not imply independent evidence. Agents can share a base model, examples, and incorrect assumptions. Conversely, two components using the same model can contribute different evidence if one checks a formal constraint or queries an independent source. Evaluations should consequently compare a multi-agent system with simpler workflows under matched information and budget. Useful controls include one agent with the combined context, repeated sampling followed by selection, and a deterministic checker for the property assigned to the reviewer.

AdverTest pairs test and mutant agents in an adversarial generation loop [6]. Such inference-time competition changes the current artifacts. It differs from reinforcement learning that changes model

Table 2. Representative mechanisms and the interpretation of their feedback. The last column describes the immediate evidence supplied by the signal, not the full set of claims in each paper.

Representative work	Main operation or feedback	Immediate interpretation
SymPrompt [40]	Path-oriented prompt decomposition	Guidance toward a structural target
CODAMOSA [27]	LLM candidates during search plateaus	New material for continued exploration
CoverUp [38]	Coverage and execution feedback	Reachability and artifact execution
MuTAP [11]	Surviving mutants guide prompts	Discrimination within a mutation model
TestGen-LLM [2]	Pass, stability, and coverage filters	Suitability for regression augmentation
Nexus [21]	Candidate execution and agent discussion	Executed behavior plus model interpretation
AdverTest [6]	Test–mutant interaction at inference	Adaptation to the current adversary
Bulbasaur [52]	Branch-guided mutator synthesis	Reusable input exploration capability

parameters across tasks, discussed in Section 6. Both can expose weaknesses in an existing test suite, but their generalization claims differ. A stronger suite for the current target is evidence about search; a stronger frozen model on new tasks is evidence about learning.

Table 2 summarizes the shared lesson across these mechanisms. Additional computation becomes useful through the information it introduces and the decisions it improves. The names of prompting strategies or agent roles are therefore less informative than their connection to an executable target and a justified oracle.

6 Learning Test Generators

6.1 Supervised Learning and Training Data

Training changes the generator's behavior beyond one target or interaction. Supervised learning commonly uses code–test pairs or descriptions paired with tests. AthenaTest provides a historical example of learning unit-test generation with focal context [48]. Such data teach framework syntax and common assertion patterns, but the training label also encodes a particular developer's testing choices. Multiple valid tests can exist for the same function, and a recorded test can be incomplete or redundant. Exact reproduction of a reference test is consequently a limited measure of learned testing ability.

Data construction determines which behaviors receive supervision. Selecting only passing tests favors executable examples; selecting tests that distinguish faulty programs adds a behavioral criterion. These filters can be combined, but each depends on the program collection used to construct the data. A dataset built from one generator's faulty solutions may overrepresent that generator's mistakes. Evaluating on different projects and independently constructed faults is necessary to determine whether a learned generator has acquired broader testing capability.

6.2 Reinforcement Learning and Reward Meaning

PyTester combines supervised learning and reinforcement learning for text-to-test generation, using automatic signals that include executable reference information [46]. RLSQM learns from automatic feedback associated with unit-test quality [43]. These approaches illustrate why the reward must be described in terms of the property being optimized. Syntax, execution success, structural coverage, maintainability, and semantic validity are related engineering concerns, but a reward for one does not directly establish improvement in another.

Reference-based rewards introduce a useful distinction between *access* and *assumption*. The generator may not read the reference code at inference, yet training can still depend on executing

Table 3. Selected information-access configurations. Entries describe the cited configuration or stage; they are not claims about every variant or the underlying model's pretraining data.

Configuration	Generation context	Additional evidence source	Consequence for interpretation
TestGen-LLM [2]	Existing tests and program context	Execution, stability, and coverage checks	Preserves observed regression behavior
AutoUAT / Test Flow [17]	User stories, then scenarios and HTML	Human assessment in the industrial study	Design and implementation stages have different access
Nexus [21]	Task information and agent interaction	Candidate implementation execution	Observations depend on candidate behavior
PyTester [46]	Description and signature at inference	Reference execution during training	Inference access does not describe supervision needs
UTRL [26]	Task-conditioned test generation	Reference and sampled programs during training	Adversarial learning retains a reference dependency
LIBRO [24]	Bug report and target-project context	Reproduction-oriented execution checks	Reported failure information is available before generation

that reference. The resulting method can be appropriate when reliable training solutions are available. Its supervision requirements differ from a method trained without them. Moreover, passing a reference assumes that the reference satisfies the relevant requirement and that the test does not depend on incidental behavior outside the contract.

Table 3 contrasts representative configurations. It shows why a single black-box or white-box label is often insufficient: information may arrive during training, through a later script-generation stage, or through an execution tool. The relevant unit of comparison is the complete configuration that produces and accepts the test.

Reward design should also account for the interaction between validity and strength. A test can obtain a high apparent defect-detection score by rejecting many programs indiscriminately. A constrained formulation would improve detection subject to a bound on rejection of legal implementations. Where legal implementations are unavailable, reference consistency is a practical proxy whose scope should be stated. This framing makes explicit which additional evidence would be needed to strengthen a claim of oracle correctness.

6.3 Adversarial Learning and Self-Play

UTRL learns a test generator through interaction with a code generator and uses reference and sampled programs in its training procedure [26]. ATGen develops adversarial reinforcement learning for test-case generation with a changing adversarial curriculum [28]. Sol-Ver studies self-play for solving and verifying code tasks [29]. TCS separates stages aimed at reference-consistent tests and adversarial testing [54]. Together, these approaches move test generation from imitating recorded tests toward learning from the behavior of generated programs.

The central opportunity is adaptive difficulty: the generator can encounter errors that remain challenging for its current capability. The corresponding evaluation question is whether this adaptation transfers beyond the opponent and task distribution used during training. A convincing analysis would compare a frozen learned generator against the same independently held-out code pool, include opponents from other model families, and report test validity alongside detection.

Co-evolving opponents should additionally be compared with fixed or periodically refreshed error pools under the same training budget. These controls isolate the value of interaction from simply seeing more erroneous programs.

Self-play also places pressure on the source of labels. Mutual agreement can select consistent code–test pairs, yet consistent pairs can share an incorrect interpretation. External requirements, trusted execution, verified constraints, and human judgments offer different ways to anchor the loop. The choice determines what can be learned reliably from the generated interaction, and it should remain visible in comparisons of supposedly autonomous methods.

6.4 Learning Inputs and Learning for Downstream Use

SAGE and LogiCase use grammar-based representations to connect input specifications to generated test inputs [1, 45]. Their contribution concerns admissible and diverse inputs. An evaluator still needs a mechanism for deciding whether the resulting program behavior is correct. This division can be valuable because input validity has a different structure from expected-output reasoning and can sometimes be checked mechanically.

UTGen targets tests useful for debugging, whereas CodeRM trains and scales tests to support code reward modeling [32, 39]. These objectives suggest an additional evaluation layer: the same test generator should be assessed both as a testing component and inside its intended downstream workflow. Improved code selection can arise from a better ranking signal even if some tests remain incorrect. Conversely, a semantically correct but weak test suite may have little effect on selection among already plausible candidates. Reporting both layers explains how the generated tests contribute to the final outcome.

7 Benchmarks and Quality Evidence

7.1 A Quality Profile Rather Than a Single Score

We organize test quality as a profile of complementary properties. *Input validity* asks whether the input satisfies the task's domain constraints. *Executability* asks whether the test can be built and run in the intended environment. *Oracle correctness* asks whether its judgment follows the intended specification. *Defect detection* asks whether it rejects relevant invalid behavior. *Non-rejection* asks whether it accepts permitted behavior. Coverage, stability, maintainability, and cost add further dimensions needed for deployment.

These properties have different denominators. Compilation success may be computed over all generated files; coverage may be computed after invalid files are discarded; mutation score may use only the retained tests. A generator producing a small usable fraction can appear strong when quality is reported only on survivors. Evaluations should therefore report the generation funnel, including attempted artifacts, executable artifacts, accepted artifacts, and final suites. Per-test and per-suite results should both be retained because one invalid assertion can make an otherwise useful suite reject a correct implementation.

Figure 3 summarizes why no single metric resolves all these questions. Coverage can locate untested code without determining whether the observations are meaningful. Mutation testing makes faults explicit but inherits the mutation operators and test-validity assumptions. Reference execution supplies concrete expected values but leaves the reference's relationship to the requirement as an external assumption. Assertion exact match adds evidence about similarity to a recorded test, while downstream code-selection accuracy measures utility in a particular consumer. Repeated-run pass rate measures stability in the repeated environment. Each remains useful when interpreted at the level it supports.

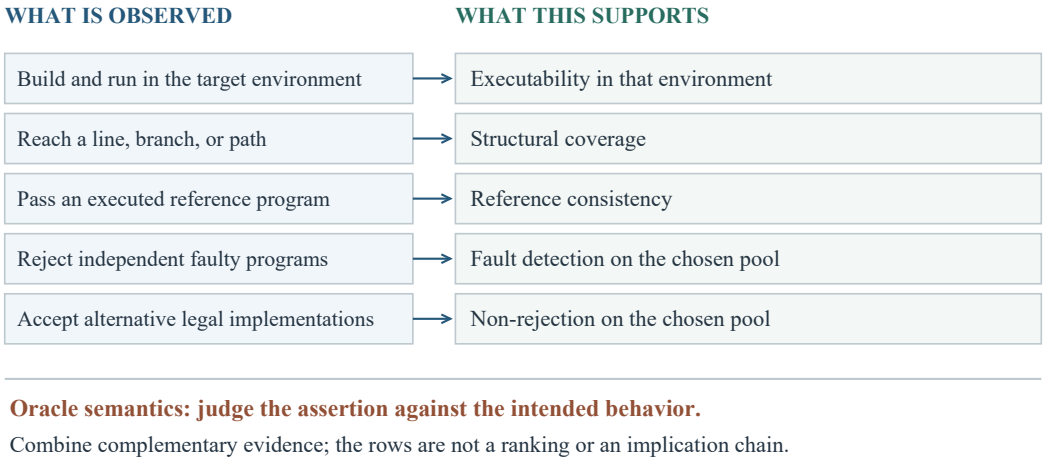


Fig. 3. Quality evidence supports different conclusions. Each arrow connects an observation to its immediate interpretation. Oracle correctness additionally depends on the intended specification and the validity of the evidence sources. These dimensions are complementary rather than successive levels of one score.

7.2 Benchmark Families

TestGenEval evaluates test generation and completion in real Python repositories [23]. TESTEVAL emphasizes test generation for program coverage, including targeted structural objectives [51]. Both are relevant to executable testing, but their task construction changes the context and the question being measured. Repository tasks expose dependencies and established tests; structural targets isolate whether generation can reach particular code regions.

SWT-Bench evaluates tests that distinguish buggy software from real-world fixes [35]. The buggy/fixed pairing gives a concrete behavioral contrast. Its interpretation depends on whether the failure is caused by the issue itself and whether the test avoids unrelated breakage. A test can distinguish one patch pair without specifying every legal implementation of the intended repair. This makes patch discrimination a valuable, bounded form of evidence.

ULT/PLT studies test generation from real-world functions and examines contamination-related evaluation conditions [22]. RESTestBench studies REST API tests generated from natural-language requirements, including requirement precision and SUT feedback [25]. These benchmarks expose issues that small function tasks can underrepresent: complex context, ambiguity, dependencies between operations, and the quality of feedback used to revise a test.

Recent empirical work adds complementary evaluation dimensions. A study of real Python bugs compares LLM and human unit tests through fault detection [49]. Research under software evolution studies test generation across semantic changes [19]. Database testing work investigates flakiness in industrial and open-source settings [5]. These studies broaden the target from initial generation quality to the behavior of tests on historical defects, changed software, and repeated executions.

Table 4 suggests a portfolio approach. A claim about repository usability needs repository evidence; a claim about requirements reasoning needs externally specified behavior; a claim about unknown-defect discovery needs independent faults. Combining these dimensions within one benchmark would be valuable, but combining headline scores from unrelated protocols would not create that evidence.

Table 4. Benchmark selection should follow the intended claim. Rows identify primary emphases rather than exhaustive benchmark capabilities.

Benchmark family	Useful question	Comparison condition to preserve
TestGenEval [23]	Can tests be constructed in repositories?	Existing tests, dependencies, generation versus completion
TESTEVAL [51]	Can tests reach structural targets?	Target visibility and coverage denominator
SWT-Bench [35]	Can tests expose an issue and accept its fix?	Issue information and buggy/fixed execution protocol
ULT/PLT [22]	How do real functions and leakage conditions affect evaluation?	Dataset construction and model exposure assumptions
RESTestBench [25]	How do requirements and SUT feedback affect API tests?	Requirement precision and correctness of feedback source

7.3 Independent Oracle Assessment

The requirement can be viewed as defining a set of permitted implementations. For an illustrative evaluation, let $\mathcal{P}^+$ be independently validated implementations that satisfy the requirement and let $\mathcal{P}^-$ be independently constructed faulty implementations. For a test suite T, define

$$\text{RejectLegal}(T) = \frac{\sum_{p \in \mathcal{P}^+} \mathbf{1}[T(p) = \text{fail}]}{|\mathcal{P}^+|}, \qquad \text{DetectFault}(T) = \frac{\sum_{p \in \mathcal{P}^-} \mathbf{1}[T(p) = \text{fail}]}{|\mathcal{P}^-|}. \quad (1)$$

The two quantities measure complementary behavior on finite evaluation pools. They require a nonempty pool of each kind, valid execution environments, and trustworthy pool labels. They are an evaluation recommendation rather than measurements extracted from the reviewed studies. Reporting them together exposes suites that detect many faults by also rejecting permitted behavior. Neither quantity alone proves specification-wide correctness.

The construction of $\mathcal{P}^+$ is especially consequential. Alternative correct implementations can vary in algorithm, data structure, permissible output order, and other underspecified choices. They test whether an oracle has encoded implementation details that the requirement does not mandate. The faulty pool should include mistakes that were not used in training, prompt construction, or feedback. Otherwise the evaluation can measure adaptation to a known pool rather than independent detection.

For natural-language scenarios, executable pools may not yet exist. Domain-expert assessment can instead evaluate traceability, applicability, expected results, and omitted behaviors. Such judgments should report the rubric, disagreement resolution, and whether assessors knew the generating method. Textual similarity to a reference scenario measures phrasing and content overlap; it does not directly establish that the scenario would expose a fault.

7.4 Comparability, Leakage, and Cost

Fair comparisons need a shared information contract. A generator that sees a reference implementation should be distinguished from one that sees only a requirement; a method that repairs tests against the candidate should be distinguished from one evaluated without candidate feedback. Training data, retrieved examples, existing test files, and evaluation artifacts can all carry information across the intended boundary. Contamination controls should state which paths are addressed instead of treating one dataset construction rule as evidence that all leakage is absent.

Cost should include model calls, input and output tokens, conventional executions, environment preparation, and human intervention. For trained generators, training cost and deployment

cost answer different questions and should be reported separately. For hybrid fuzzers, execution throughput and the reuse of generated infrastructure matter. For agent systems, total calls across all roles matter. An equal-token comparison and an equal-wall-time comparison may favor different designs, so both can be informative when resources permit.

Stochastic generation and testing require uncertainty estimates. Repeated model runs can change which tests are produced; repeated test executions can expose nondeterminism in an already fixed suite. These are distinct sources of variance. Reporting outcomes across projects and seeds, preserving failed attempts, and stating the stopping rule makes it possible to assess whether an apparent improvement reflects a broadly useful mechanism or a small set of favorable cases.

8 Practical Integration and Research Agenda

8.1 Choose a Workflow from the Testing Objective

Workflow selection starts with the behavioral decision a test should support, the available sources of intended behavior, and the missing artifact. These choices determine suitable generation mechanisms and acceptance checks.

For regression augmentation, existing tests, repository analysis, and coverage feedback can provide a productive starting point. TestGen-LLM offers an industrial example of integrating generated improvements through operational filters [2]. For conformance testing, external requirements and oracle provenance become central. For fuzzing, reusable generators and suitable failure detectors may matter more than natural-language expected outputs. For interactive code generation, user-confirmed examples can resolve ambiguity that execution alone cannot settle, as illustrated by TiCoder [16].

Invalid tests can block builds or prompt changes to correct code, while weak tests can add little detection ability. Preserving each assertion's supporting requirement or observation helps review acceptance decisions and maintain the test as software changes.

8.2 Industrial Adoption and Human Revision

Industrial evidence includes different acceptance decisions. TestGen-LLM studies improvement of existing tests through operational filtering [2]; the acceptance-testing case study evaluates both generated scenarios and their conversion into executable scripts [17]. RAGTAG and system-level test-specification generation additionally involve domain interpretation and assessment [3, 34]. A scenario judged useful by an engineer and a script accepted into a regression workflow are distinct outcomes.

A deployment evaluation should retain these stages: generated, reviewed, revised, executed, and adopted. Human effort includes interpreting requirements, supplying missing context, correcting assertions, and maintaining the accepted test. Reporting the time and rejection reasons at each stage would connect the artifact's technical quality to its actual workflow value. The reviewed examples motivate these dimensions; this synthesis does not pool their heterogeneous adoption measures.

8.3 Oracle Provenance and Independent Validation

A promising research direction is to make oracle provenance explicit in the generated artifact. An assertion could carry a link to a requirement clause, a validated example, a formal property, or a documented regression observation. Such links would help reviewers distinguish a semantic claim from an implementation-derived snapshot. They could also guide repair: changing a fixture and changing a requirement-backed assertion would trigger different validation steps.

The research question is whether provenance improves correctness and maintenance decisions. A controlled study could compare the same generator with and without provenance requirements while holding generation budget constant. Evaluation would measure unsupported assertions, rejection of legal implementations, independent fault detection, and human review time. Merely generating plausible explanatory text would not answer the question; the evidence links must be checked against their sources.

8.4 Feedback That Preserves Valid Tests

Current feedback mechanisms motivate a more precise objective for test repair: recover invalid tests while preserving valid ones. A useful evaluation would branch from the same initial test and apply alternative feedback policies. It would separately count invalid-to-valid recovery and valid-to-invalid regression, then assess the final suites on independent implementations. This design can distinguish a policy that fixes setup errors from one that improves passing rates by weakening assertions.

Possible mechanisms include separating setup repair from oracle revision, querying a reference only for disputed behavior, and asking a human to resolve genuine requirement ambiguity. The appropriate choice depends on the available evidence and its cost. The decisive comparison is against simpler policies, including no semantic repair and direct reference checking, rather than against an unconstrained baseline with less information.

8.5 Generalization in Learned and Adversarial Testers

Adversarial learning offers a route to test generators that adapt to difficult errors [26, 28, 54]. A central open question is whether the learned capability survives a change of code generator, domain, repository, or specification style. Evaluation should freeze the tester before exposing it to new tasks and separate those tasks from any adversarial curriculum construction. This would distinguish transferable capability from continued search over the evaluation instances.

Training research also needs a clearer account of test-level credit. A suite can contain both useful and misleading tests, while its aggregate reward hides which assertion caused a gain or failure. Comparing suite-level rewards with verified test-level feedback could reveal whether finer attribution improves learning. Such an investigation should retain final validity constraints and independent detection, because optimizing a more detailed proxy can still optimize the wrong property.

8.6 Repository Scale and Lifecycle Evaluation

Repository-scale evaluation should include environment construction, integration with existing tests, and maintenance across changes. TestGenEval, CITYWALK, and service-level testing provide complementary starting points [23, 36, 60]. The next step is to evaluate an artifact over time: whether it remains executable, whether its oracle should change, and whether its provenance supports that decision.

Software evolution introduces two different cases. A semantics-preserving change should generally preserve a functional oracle. A deliberate requirements change can make an old oracle obsolete. Studies of test generation under evolution and of database-test flakiness show why these dimensions need explicit treatment [5, 19]. A lifecycle benchmark should label the intended behavioral change, retain prior test versions, and measure maintenance effort together with detection of unintended regressions.

The same principle applies to context reuse. Existing tests are valuable examples, but they can carry unstable timing assumptions, brittle fixtures, or outdated requirements. Research should

investigate which context should be inherited and which should be revalidated. Comparing unrestricted retrieval with version-aware and requirement-aware retrieval would help determine whether additional context improves long-term reliability.

8.7 Reproducible Evidence and Efficient Allocation

Reproducibility packages should expose the path from request to retained suite. Relevant records include model and prompt versions, context selection, generated candidates, tool feedback, repairs, rejection reasons, and final evaluation inputs. This enables analysis of why one method succeeds and whether its filters hide a large failure burden. For training methods, data construction and reward implementation are equally important.

Budget allocation is another cross-cutting research question. A fixed budget can be spent on generating more candidates, checking oracles, executing additional faults, or resolving uncertain requirements. Dynamic scaling in CodeRM [32] and reusable fuzzing infrastructure [8, 31, 52] illustrate different responses to this allocation problem. A broader study could compare these investments under a common deployment objective, such as maximizing independent detection while bounding invalid-suite rate and latency.

9 Threats to Validity

The collection supports a qualitative map of research directions rather than estimates of their prevalence. Web-based discovery, English-language emphasis, and the stated source restrictions can omit relevant studies. Performance, accessibility, concurrency, and distributed-system testing received less focused coverage than functional testing and fuzzing. Selected neural predecessors provide context but do not constitute a complete history of learned test generation.

Inspection depth also varies across records. Abstract-level descriptions establish broad mechanisms, whereas detailed context permissions, training data, and evaluation denominators require full-text coding. The present synthesis uses those limits to constrain its claims: it reports mechanisms and analytical relationships without ranking methods by numerical gains. A systematic extension should apply a fixed multi-database protocol, complete variant-level extraction, and independent validation of the coding.

Finally, the framework is an analytical organization of the reviewed work. Its usefulness for reproducible classification has not yet been tested through an inter-rater study. Some systems naturally occupy several categories, particularly when they combine scenario generation, execution, and oracle repair. Recording variants and stages separately can preserve these distinctions better than forcing each paper into a single category.

Preprint revisions and changing model services can alter both method descriptions and reproducibility conditions. Publication bias can also favor successful applications over unsuccessful deployments. The dated source register supports version tracking, but it cannot remove these biases; updated searches and explicit treatment of negative evidence remain necessary for a systematic extension.

10 Conclusion

LLM-based test generation spans scenarios, executable suites, oracles, and reusable fuzzing infrastructure. We connected these artifacts to their information sources, generation mechanisms, and evaluation assumptions. The synthesis separates executable construction from oracle justification: coverage, reference consistency, patch discrimination, independent detection, and acceptance of legal behavior supply complementary evidence. Making information access, evidence provenance, and cost explicit supports more interpretable comparisons and guides research on generators that remain useful across repositories, adversaries, and software changes.

Data Availability

The source register, search notes, synthesis outline, bibliography, and manuscript sources accompany this draft as local supplementary materials. The register includes stable source links and inspection-depth annotations. No experimental dataset or reproduced performance results are claimed. A public archival release has not been created.

References

[1] Aditi, Hyunwoo Park, Sicheol Sung, Yo-Sub Han, and Sang-Ki Ko. 2025. SAGE: Specification-Aware Grammar Extraction for Automated Test Case Generation with LLMs. arXiv:2506.11081. https://arxiv.org/abs/2506.11081

[2] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Beliz Gokkaya, Mark Harman, Inna Harper, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated Unit Test Improvement using Large Language Models at Meta. arXiv:2402.09171. https://arxiv.org/abs/2402.09171

[3] Chetan Arora, Tomas Herda, and Verena Homm. 2024. Generating Test Scenarios from NL Requirements using Retrieval-Augmented LLMs: An Industrial Study. In *2024 IEEE 32nd International Requirements Engineering Conference (RE)*. IEEE, 240–251. doi:10.1109/RE59067.2024.00031

[4] Cristian Augusto, Antonia Bertolino, Guglielmo De Angelis, Francesca Lonetti, and Jesús Morán. 2025. Large Language Models for Software Testing: A Research Roadmap. arXiv:2509.25043. https://arxiv.org/abs/2509.25043

[5] Alexander Berndt, Thomas Bach, Rainer Gemulla, Marcus Kessel, and Sebastian Baltes. 2026. On the Flakiness of LLM-Generated Tests for Industrial and Open-Source Database Management Systems. arXiv:2601.08998. https://arxiv.org/abs/2601.08998

[6] Pengyu Chang, Yixiong Fang, Silin Chen, Yuling Shi, Beijun Shen, and Xiaodong Gu. 2026. Test vs Mutant: Adversarial LLM Agents for Robust Unit Test Generation. arXiv:2602.08146. https://arxiv.org/abs/2602.08146

[7] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. arXiv:2207.10397. https://arxiv.org/abs/2207.10397

[8] Chuyang Chen, Brendan Dolan-Gavitt, and Zhiqiang Lin. 2025. ELFuzz: Efficient Input Generation via LLM-driven Synthesis Over Fuzzer Space. In *34th USENIX Security Symposium*. USENIX Association, Seattle, WA, 6279–6298. https://www.usenix.org/conference/usenixsecurity25/presentation/chen-chuyang

[9] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: A Framework for LLM-Based Test Generation. arXiv:2305.04764. https://arxiv.org/abs/2305.04764

[10] Bei Chu, Yang Feng, Kui Liu, Zhaoqiang Guo, Yichi Zhang, Hange Shi, Zifan Nan, and Baowen Xu. 2025. Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities. arXiv:2511.21382. https://arxiv.org/abs/2511.21382

[11] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2023. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. arXiv:2308.16557. https://arxiv.org/abs/2308.16557

[12] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2022. Large Language Models are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. arXiv:2212.14834. https://arxiv.org/abs/2212.14834

[13] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large Language Models are Edge-Case Fuzzers: Testing Deep Learning Libraries via FuzzGPT. arXiv:2304.02014. https://arxiv.org/abs/2304.02014

[14] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A Neural Method for Test Oracle Generation. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2130–2141. doi:10.1145/3510003.3510141

[15] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2023. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? arXiv:2310.01831. https://arxiv.org/abs/2310.01831

[16] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. arXiv:2404.10100. https://arxiv.org/abs/2404.10100

[17] Margarida Ferreira, Luis Viegas, Joao Pascoal Faria, and Bruno Lima. 2025. Acceptance Test Generation with Large Language Models: An Industrial Case Study. In *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*. IEEE, 1–11. doi:10.1109/AST66626.2025.00007

[18] Orimoloye Folorunsho and Hassan Reza. 2026. AI-Driven Test Case Generation from Natural Language Requirements: A Survey of Techniques and Research Gaps. arXiv:2606.06563. https://arxiv.org/abs/2606.06563

[19] Sabaat Haroon, Mohammad Taha Khan, and Muhammad Ali Gulzar. 2026. Evaluating LLM-Based Test Generation Under Software Evolution. arXiv:2603.23443. https://arxiv.org/abs/2603.23443

[20] Soneya Binta Hossain, Antonio Filieri, Matthew B. Dwyer, Sebastian Elbaum, and Willem Visser. 2023. Neural-Based Test Oracle Generation: A Large-scale Evaluation and Lessons Learned. arXiv:2307.16023. https://arxiv.org/abs/2307.16023

[21] Dong Huang, Mingzhe Du, Jie M. Zhang, Zheng Lin, Meng Luo, Qianru Zhang, and See-Kiong Ng. 2025. Nexus: Execution-Grounded Multi-Agent Test Oracle Synthesis. arXiv:2510.26423. https://arxiv.org/abs/2510.26423

[22] Dong Huang, Jie M. Zhang, Mark Harman, Qianru Zhang, Mingzhe Du, and See-Kiong Ng. 2025. Benchmarking LLMs for Unit Test Generation from Real-World Functions. arXiv:2508.00408. https://arxiv.org/abs/2508.00408

[23] Kush Jain, Gabriel Synnaeve, and Baptiste Rozière. 2024. TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark. arXiv:2410.00752. https://arxiv.org/abs/2410.00752

[24] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2022. Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. arXiv:2209.11515. https://arxiv.org/abs/2209.11515

[25] Leon Kogler, Stefan Hangler, Maximilian Ehrhart, Benedikt Dornauer, Roland Wuersching, and Peter Schrammel. 2026. RESTestBench: A Benchmark for Evaluating the Effectiveness of LLM-Generated REST API Test Cases from NL Requirements. arXiv:2604.25862. https://arxiv.org/abs/2604.25862

[26] Dongjun Lee, Changho Hwang, and Kimin Lee. 2026. Learning to Generate Unit Test via Adversarial Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2026/hash/efb02f96766a3b599c76852abf4d42dd-Abstract-Conference.html

[27] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *Proceedings of the International Conference on Software Engineering (ICSE)*. https://www.microsoft.com/en-us/research/publication/codamosa-escaping-coverage-plateaus-in-test-generation-with-pre-trained-large-language-models/

[28] Qingyao Li, Xinyi Dai, Weiwen Liu, Xiangyang Li, Yasheng Wang, Ruiming Tang, Yong Yu, and Weinan Zhang. 2026. ATGen: Adversarial Reinforcement Learning for Test Case Generation. In *International Conference on Learning Representations (ICLR)*. https://proceedings.iclr.cc/paper_files/paper/2026/hash/eed25c037bc08afcbefab6f7a6b700e0-Abstract-Conference.html

[29] Zi Lin, Sheng Shen, Ilia Kulikov, Jingbo Shang, Jason Weston, and Yixin Nie. 2025. Learning to Solve and Verify: A Self-Play Framework for Code and Test Generation. arXiv:2502.14948. https://arxiv.org/abs/2502.14948

[30] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. arXiv:2310.15780. https://arxiv.org/abs/2310.15780

[31] Yunlong Lyu, Yuxuan Xie, Peng Chen, and Hao Chen. 2024. Prompt Fuzzing for Fuzz Driver Generation. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. ACM, 3793–3807. doi:10.1145/3658644.3670396

[32] Zeyao Ma, Xiaokang Zhang, Jing Zhang, Jifan Yu, Sijia Luo, and Jie Tang. 2025. Dynamic Scaling of Unit Tests for Code Reward Modeling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 6917–6935. doi:10.18653/v1/2025.acl-long.343

[33] Ruijie Meng, Martin Mirchev, Marcel Böhme, and Abhik Roychoudhury. 2024. Large Language Model guided Protocol Fuzzing. In *Proceedings 2024 Network and Distributed System Security Symposium*. Internet Society. doi:10.14722/ndss.2024.24556

[34] Dragan Milchevski, Gordon Frank, Anna Hätty, Bingqing Wang, Xiaowei Zhou, and Zhe Feng. 2025. Multi-Step Generation of Test Specifications using Large Language Models for System-Level Requirements. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*. Association for Computational Linguistics, 132–146. doi:10.18653/v1/2025.acl-industry.11

[35] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. SWT-Bench: Testing and Validating Real-World Bug-Fixes with Code Agents. arXiv:2406.12952. https://arxiv.org/abs/2406.12952

[36] Rangeet Pan, Raju Pavuluri, Ruikai Huang, Rahul Krishna, Tyler Stennett, Alessandro Orso, and Saurabh SInha. 2025. SAINT: Service-level Integration Test Generation with Program Analysis and LLM-based Agents. arXiv:2511.13305. https://arxiv.org/abs/2511.13305

[37] André Pereira, Bruno Lima, and João Pascoal Faria. 2026. APITestGenie: Generating Web API Tests from Requirements and API Specifications with LLMs. arXiv:2604.02039. https://arxiv.org/abs/2604.02039

[38] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919. doi:10.1145/3729398

[39] Archiki Prasad, Elias Stengel-Eskin, Justin Chih-Yao Chen, Zaid Khan, and Mohit Bansal. 2025. Learning to Generate Unit Tests for Automated Debugging. arXiv:2502.01619. https://arxiv.org/abs/2502.01619

[40] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A study of Coverage Guided Test Generation in Regression Setting using LLM. arXiv:2402.00097. https://arxiv.org/abs/2402.00097

[41] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. arXiv:2302.06527. https://arxiv.org/abs/2302.06527

[42] Seung Yeob Shin, Fabrizio Pastore, Domenico Bianculli, and Alexandra Baicoianu. 2024. Towards Generating Executable Metamorphic Relations Using Large Language Models. arXiv:2401.17019. https://arxiv.org/abs/2401.17019

[43] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation. arXiv:2310.02368. https://arxiv.org/abs/2310.02368

[44] Tyler Stennett, Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2025. AutoRestTest: A Tool for Automated REST API Testing Using LLMs and MARL. arXiv:2501.08600. https://arxiv.org/abs/2501.08600

[45] Sicheol Sung, Aditi, Dogyu Kim, Yo-Sub Han, and Sang-Ki Ko. 2025. LogiCase: Effective Test Case Generation from Logical Description in Competitive Programming. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 7742–7750. doi:10.24963/ijcai.2025/861

[46] Wannita Takerngsaksiri, Rujikorn Charakorn, Chakkrit Tantithamthavorn, and Yuan-Fang Li. 2025. PyTester: Deep Reinforcement Learning for Text-to-Testcase Generation. *Journal of Systems and Software* 224 (2025), 112381. doi:10.1016/j.jss.2025.112381

[47] Murat Tasarsu, Ahmet Vedat Tokmak, and Cagatay Catal. 2026. Test case generation using large language models: a systematic literature review. *Cluster Computing* 29, 4, Article 227 (2026). doi:10.1007/s10586-026-06021-z

[48] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit Test Case Generation with Transformers and Focal Context. arXiv:2009.05617. https://arxiv.org/abs/2009.05617

[49] Phouvadeth Vathana, Prapti Bhatt, Rishi Patel, and Nasir U. Eisty. 2026. LLM vs. Human Unit Tests: Fault Detection on Real Python Bugs. arXiv:2606.08588. https://arxiv.org/abs/2606.08588

[50] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software Testing with Large Language Models: Survey, Landscape, and Vision. arXiv:2307.07221. https://arxiv.org/abs/2307.07221

[51] Wenhan Wang, Chenyuan Yang, Zhijie Wang, Yuheng Huang, Zhaoyang Chu, Da Song, Lingming Zhang, An Ran Chen, and Lei Ma. 2025. TESTEVAL: Benchmarking Large Language Models for Test Case Generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 3547–3562. doi:10.18653/v1/2025.findings-naacl.197

[52] Yiyi Wang, Dongsong Yu, Ruiqi Dong, Yiyang Chen, Xiaogang Zhu, and Chao Zhang. 2026. Bulbasaur: Branch-Guided Online Mutator Generation for Greybox Fuzzing. In *35th USENIX Security Symposium*. USENIX Association, Baltimore, MD, 4089–4108. https://www.usenix.org/conference/usenixsecurity26/presentation/wang-yiyi

[53] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2023. Fuzz4All: Universal Fuzzing with Large Language Models. arXiv:2308.04748. https://arxiv.org/abs/2308.04748

[54] Jiacheng Xu, Wentao Zhang, Zhiyi Lyu, Fuxiang Zhang, Chaojie Wang, Yang Liu, and Bo An. 2026. Two-Stage Reinforcement Learning for Sound and Adversarial Test Generation in Code LLMs. arXiv:2609.03955. https://arxiv.org/abs/2609.03955

[55] Qinghua Xu, Guancheng Wang, Lionel Briand, and Kui Liu. 2025. Hallucination to Consensus: Multi-Agent LLMs for End-to-End JUnit Test Generation. arXiv:2506.02943. https://arxiv.org/abs/2506.02943

[56] Zhenzhen Yang, Rubing Huang, Chenhui Cui, Nan Niu, and Dave Towey. 2025. Requirements-Based Test Generation: A Comprehensive Survey. arXiv:2505.02015. https://arxiv.org/abs/2505.02015

[57] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous Large Language Model Agents Enabling Intent-Driven Mobile GUI Testing. arXiv:2311.08649. https://arxiv.org/abs/2311.08649

[58] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. arXiv:2506.15227. https://arxiv.org/abs/2506.15227

[59] Quanjun Zhang, Ye Shang, Siqi Gu, Jianyi Zhou, Chunrong Fang, Zhenyu Chen, and Liang Xiao. 2026. Multi-Agent LLM Collaboration for Unit Test Generation via Human-Testing-Inspired Workflows. arXiv:2607.09101. https://arxiv.org/abs/2607.09101

[60] Yuwei Zhang, Qingyuan Lu, Kai Liu, Wensheng Dou, Jiaxin Zhu, Li Qian, Chunxi Zhang, Zheng Lin, and Jun Wei. 2025. CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge. arXiv:2501.16155. https://arxiv.org/abs/2501.16155